And, as in uffish thought he stood,
The Jabberwock, with eyes of flame,
Came whiffling through the tulgey wood,
And burbed as it came!

One, two! One, two! And through and through
The vorpal blade went snicker-snack!
He left it dead, and with its head
He went galumphing back.

— Lewis Carroll (in Through the Looking-Glass)

And Alice said

*Somehow it seems to fill my head with ideas—only I don't exactly know what they are! **Somebody** killed **something**: that's clear, at any rate.*

ILOs

After this class the student

- knows what LMs **are**
- can **design** LMs
- can **estimate** the parameters of LMs
- can **evaluate** LMs

NLP1

Text Generation

Wilker Aziz



https://nlp-illc.github.io/nlp1/2026/

Fall 2026

Outline

1. Language Models
2. Factorisation
3. Parameterisation
4. Parameter Estimation
5. Evaluation

Language Models

Language model

A language model is a **probability distribution** over the set of all strings in a language.

Being a probability distribution means that

1. an LM can assign probability to text
2. you can generate text by drawing samples from the LM

LMs give us a way to make computers generate linguistic data without us having to first figure out **how language works**.

Applications

- Order alternative sentences (e.g., in speech recognition).
- Generate text in context (e.g., autocomplete).
- Backbone of various NLP systems: translation, summarisation, question-answering, chatbots.
- Backbone of modern AI: tool-using, multilingual, multimodal chatbots and agents.

A word on probabilities

Probability IS NOT a property of text,[1]

- we **cannot** learn LMs by regressing from text to "observed target probabilities"

Probability is an expression of preference (by a model or observer).[2]

We **can** learn to assign probability to text by

- designing a probabilistic model of how texts come about
- and optimising the parameters of this model using observed data and a statistical criterion of our choice.

---

[1] De Finetti's seminal *Theory of Probability* starts with a provocative claim: PROBABILITY DOES NOT EXIST.

[2] On occasion, we can make it capture *sample frequency* in a population.

A Good LM is...

A probability distribution whose samples **resemble** observed text.[3]

Examples:

- if the typical sentence has 30 words, sampling from a good LM will reproduce that pattern;
- if the typical sentence has SUBJ VERB OBJ (in this order), samples from a good LM will be like that too;
- if observed texts are long and exhibit a certain coherence of topics, LM-generated text will exhibit such coherence too;
- etc.

[3] This is a statistical notion of 'goodness', we will discuss other notions later.

Designing LMs

Since an LM is a probability distribution, designing one requires choosing:

- A **sample space.** The set of outcomes that the LM can generate and assign probability to.
- A **probability mass function (pmf).** A function mapping each and every outcome in the sample space to its assigned probability mass.

Text as a Data Type

We regard text as a finite sequence of discrete symbols which we generally refer to as 'words' (or, even more generally, as 'tokens').

Digital text is a sequence of characters. Using a *tokenisation algorithm* we 'segment' digital text into a sequence of tokens.

Example: with a tokenisation procedure based on spaces and punctuation, the English text what a nice dog! is regarded as a sequence of 5 tokens: (what, a, nice, dog, !).[4]

[4] Modern, general-purpose tokenisers are based on compression algorithms sennrich-etal-2016-neural. See tutorial 1.

Sample Space

Start with a vocabulary of **words** $\mathcal{W}$ (for example, all unique tokens found in some large, representative dataset).

Then the sample space is the set of all finite-length sequences made of symbols from this vocabulary. This set, denoted $\mathcal{W}^*$, is the **language** (in a formal, non-linguistic sense).

Example

- with $\mathcal{W}$ = {a, cat, dog, nice}
- then a nice cat and a dog are sentences in the language $\mathcal{W}^*$, just like a a or nice a nice a nice, but a cute dog isn't.

Formalisation

W is a random word. An outcome w is a symbol in a vocabulary $\mathcal{W}$ of size V.

$X = \langle W_1, \ldots, W_L \rangle$ is a random *sequence* of L words. We can also denote it $W_{1:L}$. An outcome $\langle w_1, \ldots, w_l \rangle$ is a sequence in $\mathcal{W}^*$, that is, a sequence of l symbols from $\mathcal{W}$.

A language model is a mechanism to assign a probability value $p_X(w_{1:l})$ to **each and every** outcome $w_{1:l} \in \mathcal{W}^*$.

We now design a pmf to map $w_{1:l}$ to its probability mass.

The subscript in p_X reminds you of which pmf we are talking about. Think of it as part of the pmf's name. It will become useful later when we will have expressions involving multiple pmfs.

Challenge

The language is a countably infinite set of variable-length sequences.

Intuition. To see that this is a real challenge, let's list outcomes of X in a table and associate each outcome with a scalar *parameter* for the outcome's probability mass.

Unique id	Outcome x	Probability $p_X(x)$
1	nice!	θ_1
2	a cat!	θ_2
3	a cute cat!	θ_3
4	a nasty cat!	θ_4
5	what a cute cat!	θ_5
6	what a nasty cat!	θ_6
	...	

How many parameters do we need in order to specify p_X fully?

The tabular view isn't reasonable

What we just tried to use is a *tabular representation* of the pmf of a discrete random variable.

In this representation, assigning probability to x takes a simple table lookup:

$$p_X(x) = \theta_{\mathrm{id}(x)}$$

The tabular representation is based on enumeration of outcomes and its parameters are statistically independent of one another.

This is computationally and statistically inefficient, and, with countably infinite sample spaces, **this is unusable**.

Factorisation

Key Idea

Think of an outcome $w_{1:l} \in \mathcal{W}^*$ as a *decomposable structure*.

Imagine a procedure by which you could *derive* this structure in a sequence of *steps*.

We can then express the probability of any one sequence $w_{1:l}$ in $\mathcal{W}^*$ using probabilities assigned to the steps that jointly derive it.

Example – Deriving Text

Consider the sequence $x = \langle \text{He, went, to, the, store, EOS} \rangle$.[6] Now think of x as the result of incrementally expanding an empty sequence, one symbol at a time. Start with $h_1 = \langle\rangle$, then

1. Given h_1, choose the first word: He. Make $h_2 \leftarrow h_1 \circ \langle \text{He} \rangle$.

[6] EOS marks the end of the sequence, the need for it will become clear.

Example – Deriving Text

Consider the sequence $x = \langle \text{He, went, to, the, store, EOS} \rangle$.[6] Now think of x as the result of incrementally expanding an empty sequence, one symbol at a time. Start with $h_1 = \langle\rangle$, then

1. Given h_1, choose the first word: He. Make $h_2 \leftarrow h_1 \circ \langle \text{He} \rangle$.
2. Given h_2, choose the second word: went. Make $h_3 \leftarrow h_2 \circ \langle \text{went} \rangle$.

[6] EOS marks the end of the sequence, the need for it will become clear.

Example – Deriving Text

Consider the sequence $x = \langle \text{He, went, to, the, store, EOS} \rangle$.[6] Now think of x as the result of incrementally expanding an empty sequence, one symbol at a time. Start with $h_1 = \langle\rangle$, then

1. Given h_1, choose the first word: He. Make $h_2 \leftarrow h_1 \circ \langle \text{He} \rangle$.
2. Given h_2, choose the second word: went. Make $h_3 \leftarrow h_2 \circ \langle \text{went} \rangle$.
3. Given h_3, choose the third word: to. Make $h_4 \leftarrow h_3 \circ \langle \text{to} \rangle$.

[6] EOS marks the end of the sequence, the need for it will become clear.

Example – Deriving Text

Consider the sequence $x = \langle \text{He, went, to, the, store, EOS} \rangle$.[6] Now think of x as the result of incrementally expanding an empty sequence, one symbol at a time. Start with $h_1 = \langle\rangle$, then

1. Given h_1, choose the first word: He. Make $h_2 \leftarrow h_1 \circ \langle \text{He} \rangle$.
2. Given h_2, choose the second word: went. Make $h_3 \leftarrow h_2 \circ \langle \text{went} \rangle$.
3. Given h_3, choose the third word: to. Make $h_4 \leftarrow h_3 \circ \langle \text{to} \rangle$.
4. Given h_4, choose the fourth word: the. Make $h_5 \leftarrow h_4 \circ \langle \text{the} \rangle$.

[6] EOS marks the end of the sequence, the need for it will become clear.

Example – Deriving Text

Consider the sequence $x = \langle \text{He, went, to, the, store, EOS} \rangle$.[6] Now think of x as the result of incrementally expanding an empty sequence, one symbol at a time. Start with $h_1 = \langle\rangle$, then

1. Given h_1, choose the first word: He. Make $h_2 \leftarrow h_1 \circ \langle \text{He} \rangle$.
2. Given h_2, choose the second word: went. Make $h_3 \leftarrow h_2 \circ \langle \text{went} \rangle$.
3. Given h_3, choose the third word: to. Make $h_4 \leftarrow h_3 \circ \langle \text{to} \rangle$.
4. Given h_4, choose the fourth word: the. Make $h_5 \leftarrow h_4 \circ \langle \text{the} \rangle$.
5. Given h_5, choose the fifth word: store. Make $h_6 \leftarrow h_5 \circ \langle \text{store} \rangle$.

[6] EOS marks the end of the sequence, the need for it will become clear.

Example – Deriving Text

Consider the sequence $x = \langle \text{He, went, to, the, store, EOS} \rangle$.[6] Now think of x as the result of incrementally expanding an empty sequence, one symbol at a time. Start with $h_1 = \langle\rangle$, then

1. Given h_1, choose the first word: He. Make $h_2 \leftarrow h_1 \circ \langle \text{He} \rangle$.
2. Given h_2, choose the second word: went. Make $h_3 \leftarrow h_2 \circ \langle \text{went} \rangle$.
3. Given h_3, choose the third word: to. Make $h_4 \leftarrow h_3 \circ \langle \text{to} \rangle$.
4. Given h_4, choose the fourth word: the. Make $h_5 \leftarrow h_4 \circ \langle \text{the} \rangle$.
5. Given h_5, choose the fifth word: store. Make $h_6 \leftarrow h_5 \circ \langle \text{store} \rangle$.
6. Given h_6, choose the sixth word: EOS. Make $x = h_6 \circ \langle \text{EOS} \rangle$.

[6] EOS marks the end of the sequence, the need for it will become clear.

Example – Deriving Text

Consider the sequence $x = \langle \text{He, went, to, the, store, EOS} \rangle$.[6] Now think of x as the result of incrementally expanding an empty sequence, one symbol at a time. Start with $h_1 = \langle\rangle$, then

1. Given h_1, choose the first word: He. Make $h_2 \leftarrow h_1 \circ \langle \text{He} \rangle$.
2. Given h_2, choose the second word: went. Make $h_3 \leftarrow h_2 \circ \langle \text{went} \rangle$.
3. Given h_3, choose the third word: to. Make $h_4 \leftarrow h_3 \circ \langle \text{to} \rangle$.
4. Given h_4, choose the fourth word: the. Make $h_5 \leftarrow h_4 \circ \langle \text{the} \rangle$.
5. Given h_5, choose the fifth word: store. Make $h_6 \leftarrow h_5 \circ \langle \text{store} \rangle$.
6. Given h_6, choose the sixth word: EOS. Make $x = h_6 \circ \langle \text{EOS} \rangle$.

Assign probability to $X = x$ by assigning probability to each of these 'steps', all of which must be taken in order to reproduce x.

[6] EOS marks the end of the sequence, the need for it will become clear.

Example – Assigning Probability

$$
\begin{aligned}
&p_X(\langle \text{He, went, to, the, store} \rangle) \\
&\quad p_{W|H}(\text{He}|\langle\rangle) \\
&\qquad \times p_{W|H}(\text{went}|\langle \text{He} \rangle) \\
&\qquad \times p_{W|H}(\text{to}|\langle \text{He, went} \rangle) \\
&\qquad \times p_{W|H}(\text{the}|\langle \text{He, went, to} \rangle) \\
&\qquad \times p_{W|H}(\text{store}|\langle \text{He, went, to, the} \rangle) \\
&\qquad \times p_{W|H}(\text{EOS}|\langle \text{He, went, to, the, store} \rangle)
\end{aligned}
$$

We are specifying how p_X assigns probability to the text $\langle \text{He, went, to, the, store} \rangle$. We choose to express that number as a product of probabilities which some other pmfs (of the form $p_{W|H}$) assign to the words in the text as we generate it word for word. Each time, we assign probability to a word, we do it *conditioned on* an ordered **history** of words that precede it.

Designing LMs via Chain Rule of Probabilities

Our most general LM assigns probability to $w_{1:l}$ as defined below:

$$p_X(w_{1:l}) \stackrel{\text{def}}{=} \prod_{i=1}^{l} p_{W|H}(w_i \mid w_{<i}) \tag{1}$$

This is what we call an *autoregressive factorisation* of the probability of a sequence.

We will work on the design of the conditional pmfs that appear on the right-hand side of the equation.

Conditional Probability Distribution (CPD)

For any given history h, such as $\langle \text{He}, \text{went}, \text{to}, \text{the} \rangle$, we need to assign probability $p_{W|H}(w|h)$ to any symbol w in the vocabulary (there are $V = |\mathcal{W}|$ such symbols).

In tabular representation, the pmf of the Categorical random variable $W|H = h$ can be represented by a unit-norm, V-dimensional vector $\boldsymbol{\theta}^{(h)}$ of probability masses.

id	w	$p_{W\|H}(w\|h)$
1	a	$\theta_1^{(h)}$
2	amazing	$\theta_2^{(h)}$
	...	
V	zyzzyva	$\theta_V^{(h)}$

Table: *

Tabular view of $W|H = h$

Assigning Probability or Generating Outcomes

Our procedure to assign probability to an outcome also prescribes a *sampler* (or *simulator*, or *generator*), that is, an algorithm to *generate* outcomes using the LM.

This procedure is also known as a **generative story**.

Generative Story

1. Start with an empty history $h_1 = \langle\rangle$. Set $i = 1$.
2. Condition on the available history h_i and draw a word w_i with probability $p_{W|H}(w_i|h_i)$ extending the history with it.
3. If w_i is a special end-of-sequence symbol (EOS), terminate, else increment i and repeat (2).

This specifies a **factorisation** of p_X in terms of elementary factors of the kind $p_{W|H}$.

Summary

- An LM is a distribution p_X over the space of all texts
- Rather than designing p_X directly, we re-express it via chain rule
- For any given history h, we must be able to prescribe a pmf $p_{W|H=h}$ over the vocabulary
- The vocabulary is finite, so a pmf $p_{W|H=h}$ is representable by a tractable V-dimensional vector.
- There's no limit to the set of possible histories.
- So we cannot hope to store all pmfs of the kind $p_{W|H=h}$, but perhaps we can predict them when they are needed?

We deal with this next!

Parameterisation

Factorising and Parameterising

Factorising $p_X(x)$ means decomposing this quantity using a product of **elementary factors**.[7]

We have used chain rule to decompose it as $p_X(w_{1:l}) = \prod_{i=1}^{l} p_{W|H}(w|h)$.

We now design a mechanism to compute $p_{W|H}(w|h)$ for any choice of (h, w). This design is what we call a **parameterisation**.

[7] For intuition, consider this analogy. The composite number 24 can be factorised into a product of prime numbers: $2 \times 2 \times 2 \times 3 = 2^3 \times 3$. Prime numbers are 'elementary' in that they cannot be further factorised.

Major Ideas in NLP

Assign probability to any $w \in \mathcal{W}$ given any $h \in \mathcal{W}^*$

1. using the *relative frequency* of $h \circ \langle w \rangle$, as observed in a large corpus;

Major Ideas in NLP

Assign probability to any $w \in \mathcal{W}$ given any $h \in \mathcal{W}^*$

1. using the *relative frequency* of $h \circ \langle w \rangle$, as observed in a large corpus;
2. using a *softmax regressor* to map from h to V next-word probabilities.

Relative Frequency

Define probability of store given he went to the as the frequency of **he went to the store** relative to **he went to the** • in a large corpus.

Generically, for a word w and a history h:

$$p_{W|H}(w|h) \overset{\text{MLE}}{=} \frac{\text{count}_{HW}(h, w)}{\sum_{o \in \mathcal{W}} \text{count}_{HW}(h, o)} \tag{2}$$

This corresponds to maximum likelihood estimation (MLE) for tabular Categorical cpds of the kind $p_{W|H=h}$.

Relative Frequency

Define probability of store given he went to the as the frequency of **he went to the store** relative to **he went to the** • in a large corpus.

Generically, for a word w and a history h:

$$p_{W|H}(w|h) \overset{\text{MLE}}{=} \frac{\text{count}_{HW}(h, w)}{\sum_{o \in \mathcal{W}} \text{count}_{HW}(h, o)} \tag{2}$$

This corresponds to maximum likelihood estimation (MLE) for tabular Categorical cpds of the kind $p_{W|H=h}$.

Do you see any problem with this idea?

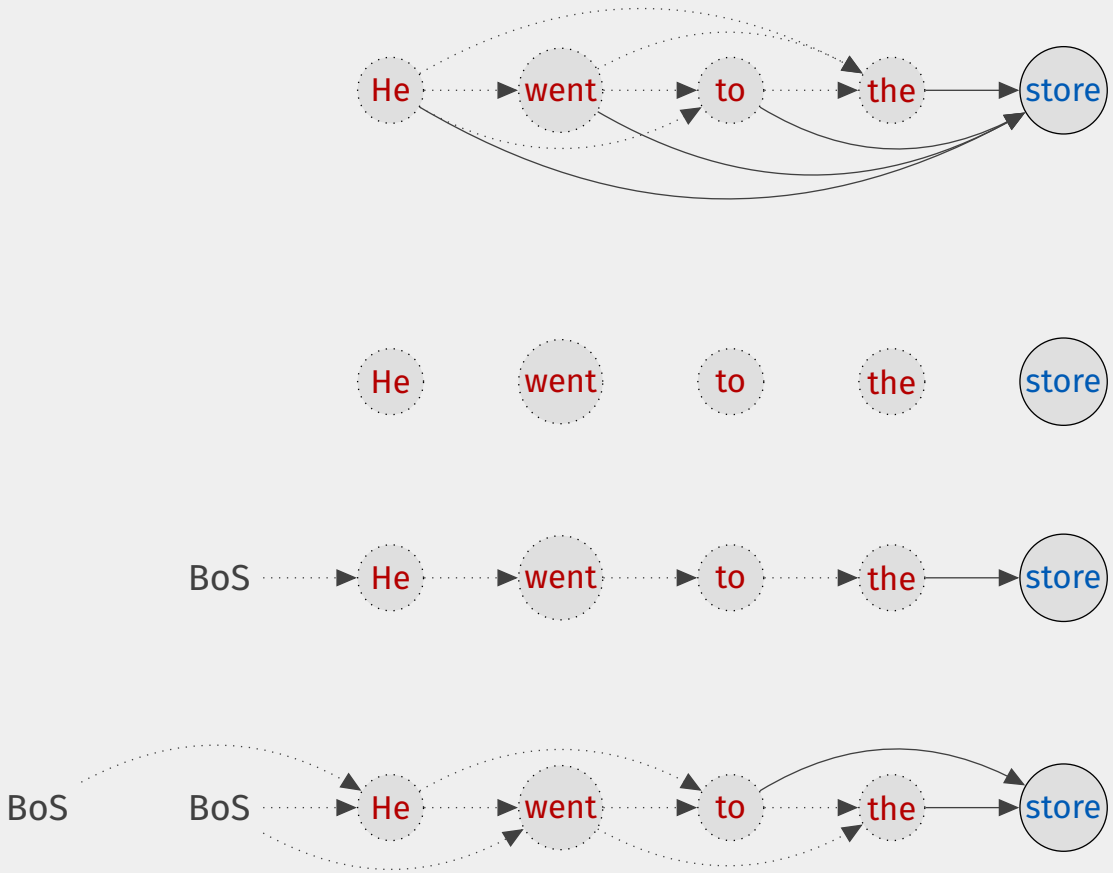
Data Sparsity!

If we have not seen a given *h* followed by a certain *w*, the relative frequency is 0. If we have not seen a given *h*, the relative frequency is not even defined.

Unavoidable truth about empirical methods: not seeing something is not evidence of it not being possible. It's just data sparsity speaking.

An answer that was popular for decades: change the **factorisation**, simplifying *h* to retain only words that are closest to the next one.

Assume that the next word is conditionally independent of all but the $N - 1$ preceding words.



From the top: autoregressive LM, unigram LM (N=1), bigram LM (N=2), trigram LM (N=3).

Cond. Independence and Markov Assumptions

The key idea in the NGram LM is to make a conditional independence assumption:

$$p_X(w_{1:\ell}) \overset{\text{ind.}}{=} \prod_{i=1}^{\ell} P_{W|H}(w_i \mid \text{last}_{N-1}(w_{<i})) \tag{3}$$

a word is independent on all but the recent history of $N - 1$ words. This is the so-called **Markov assumption** of order $N - 1$.

Note how the elementary factors of the NGram LM always depend on the same number ($N - 1$) of previous words.

Tabular CPDs for NGram LMs

Store relative frequencies of observed NGrams in a table.

Generically, for a word w, a history h and NGram size N:

$$P_{W|H}(w|h) = \frac{\text{count}_{HW}(\text{last}_{N-1}(h), w)}{\sum_{o \in \mathcal{W}} \text{count}_{HW}(\text{last}_{N-1}(h), o)} \tag{4}$$

Example: in a 3-gram LM (i.e., $N = 3$), to assign probability to store given ⟨he, went, to, the⟩, we truncate the history to the last 2 words ⟨to, the⟩ such that it has length 2 and use the frequency of **to the store** relative to the frequency of **to the** • in a large corpus.

Tabular CPDs for NGram LMs

Store relative frequencies of observed NGrams in a table.

Generically, for a word w, a history h and NGram size N:

$$P_{W|H}(w|h) = \frac{\text{count}_{HW}(\text{last}_{N-1}(h), w)}{\sum_{o \in \mathcal{W}} \text{count}_{HW}(\text{last}_{N-1}(h), o)} \tag{4}$$

Example: in a 3-gram LM (i.e., $N = 3$), to assign probability to store given ⟨he, went, to, the⟩, we truncate the history to the last 2 words ⟨to, the⟩ such that it has length 2 and use the frequency of **to the store** relative to the frequency of **to the** • in a large corpus.

Any problems with this?

Limitations

Inefficient

- increasing N by one (e.g., $N = 3 \rightarrow N = 4$) leads to exponentially many more parameters because we have V^N combinations of (h, w)

Data sparsity

- treating histories as unrelated leaves us with very sparse statistics about (h, w) co-occurrences

Unrealistic

- it excludes all long-range dependencies

Limitations

Inefficient

- increasing N by one (e.g., $N = 3 \rightarrow N = 4$) leads to exponentially many more parameters because we have V^N combinations of (h, w)

Data sparsity

- treating histories as unrelated leaves us with very sparse statistics about (h, w) co-occurrences

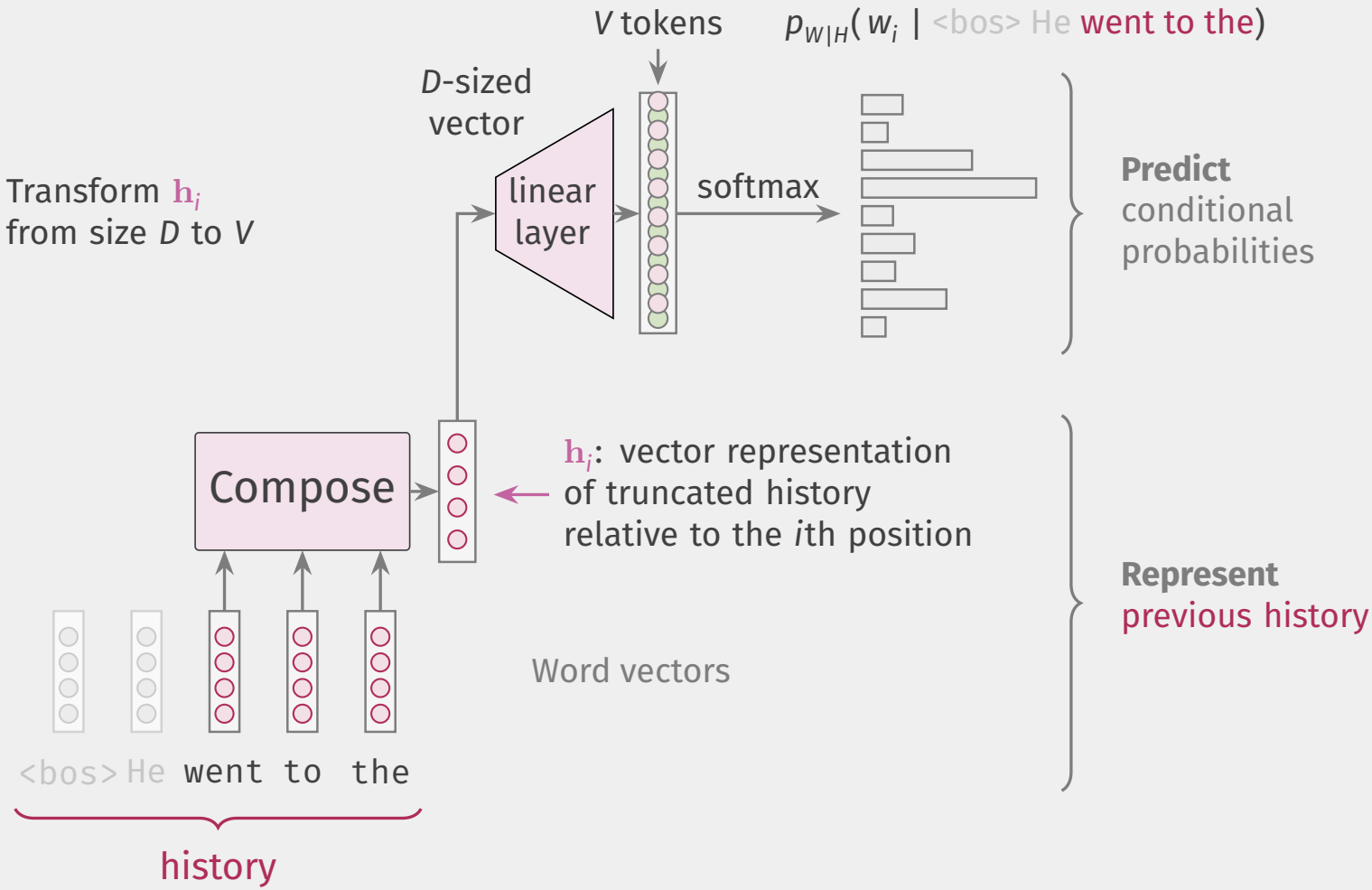
Unrealistic

- it excludes all long-range dependencies

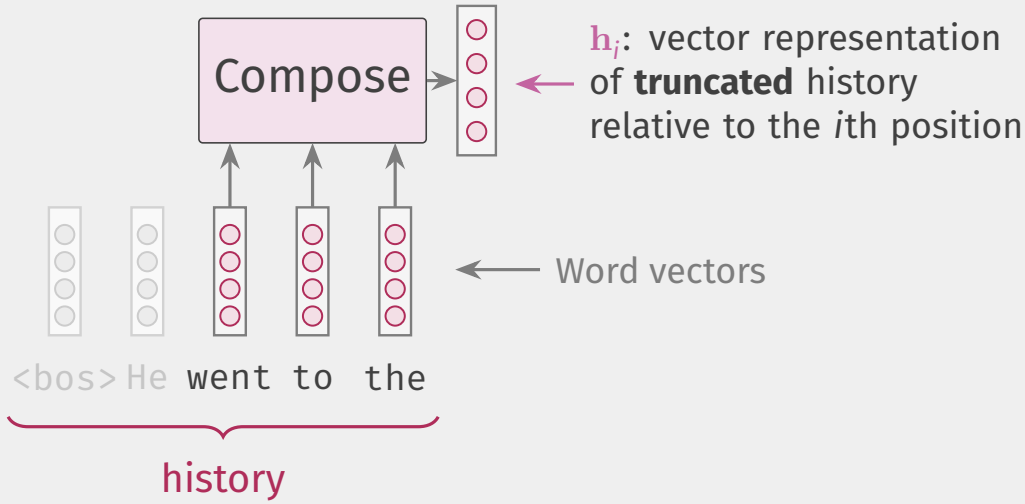
Let's first parameterise this better, then we will see an opportunity to get rid of Markov assumptions.

Next Word Probabilities with NNs

A softmax regressor that maps a truncated history of size 3 to a next-word distribution:



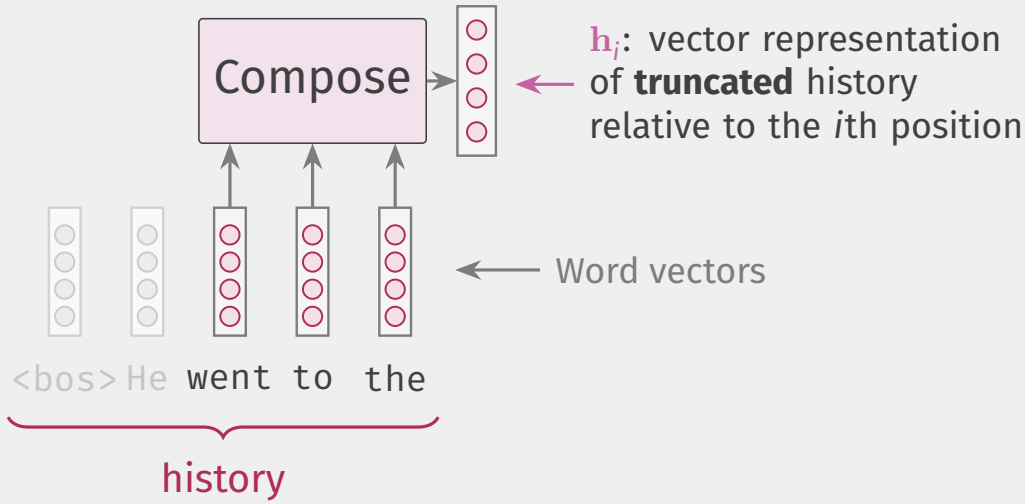
Word vectors



Represent each token (a string) as a fixed-dimensional vector:

- *V*-dimensional word indicators
- *D*-dimensional trainable vectors
  (better ideas in the next module)

Composition Function



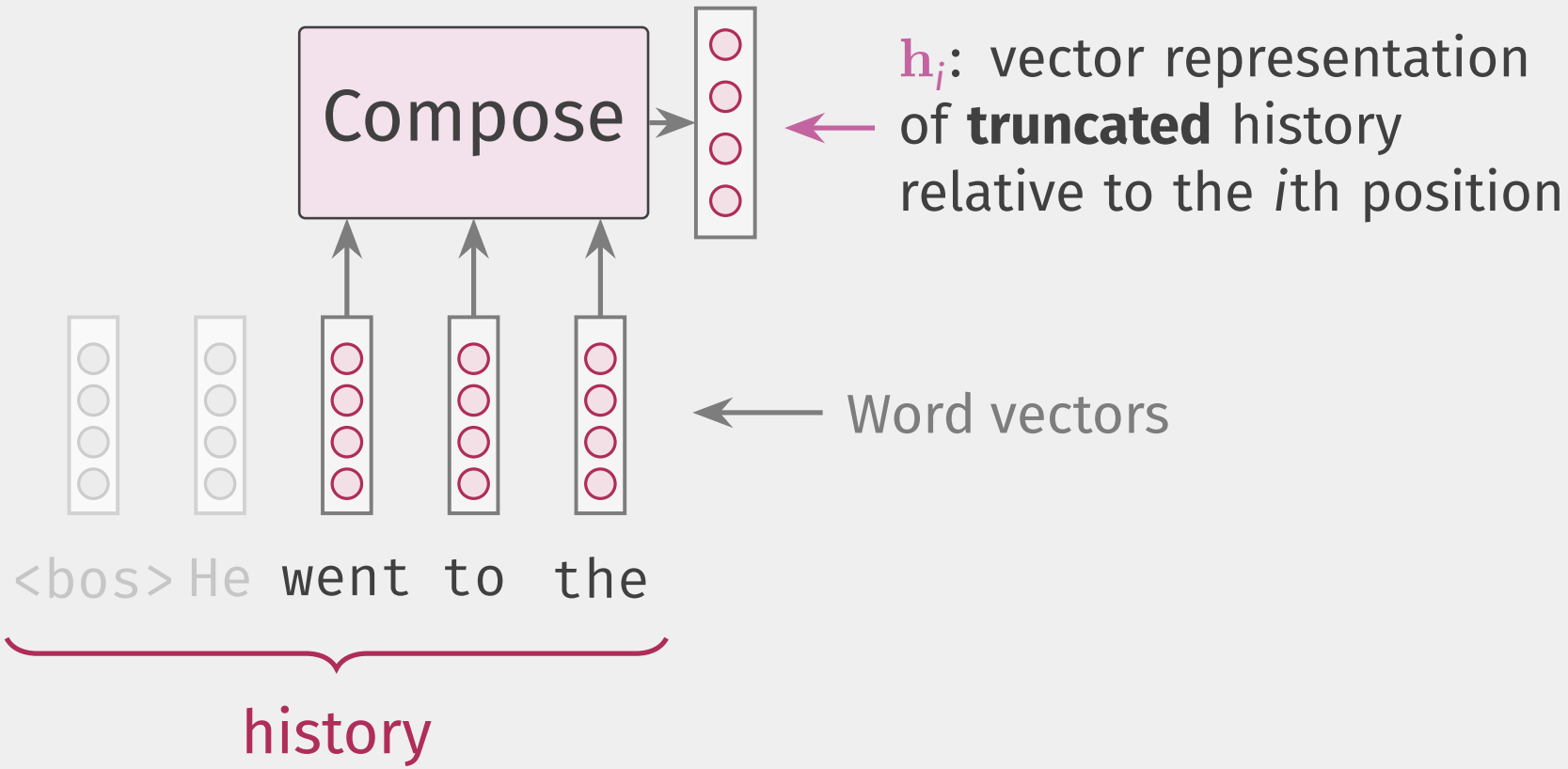
Combines a fixed number of inputs (here 3 vectors) into a single vector of fixed dimensionality:

- average;
- concatenate the inputs;
- transform inputs using a feed-forward NN (FFNN);

Discussion

This is considerably better than those tabular CPDs.

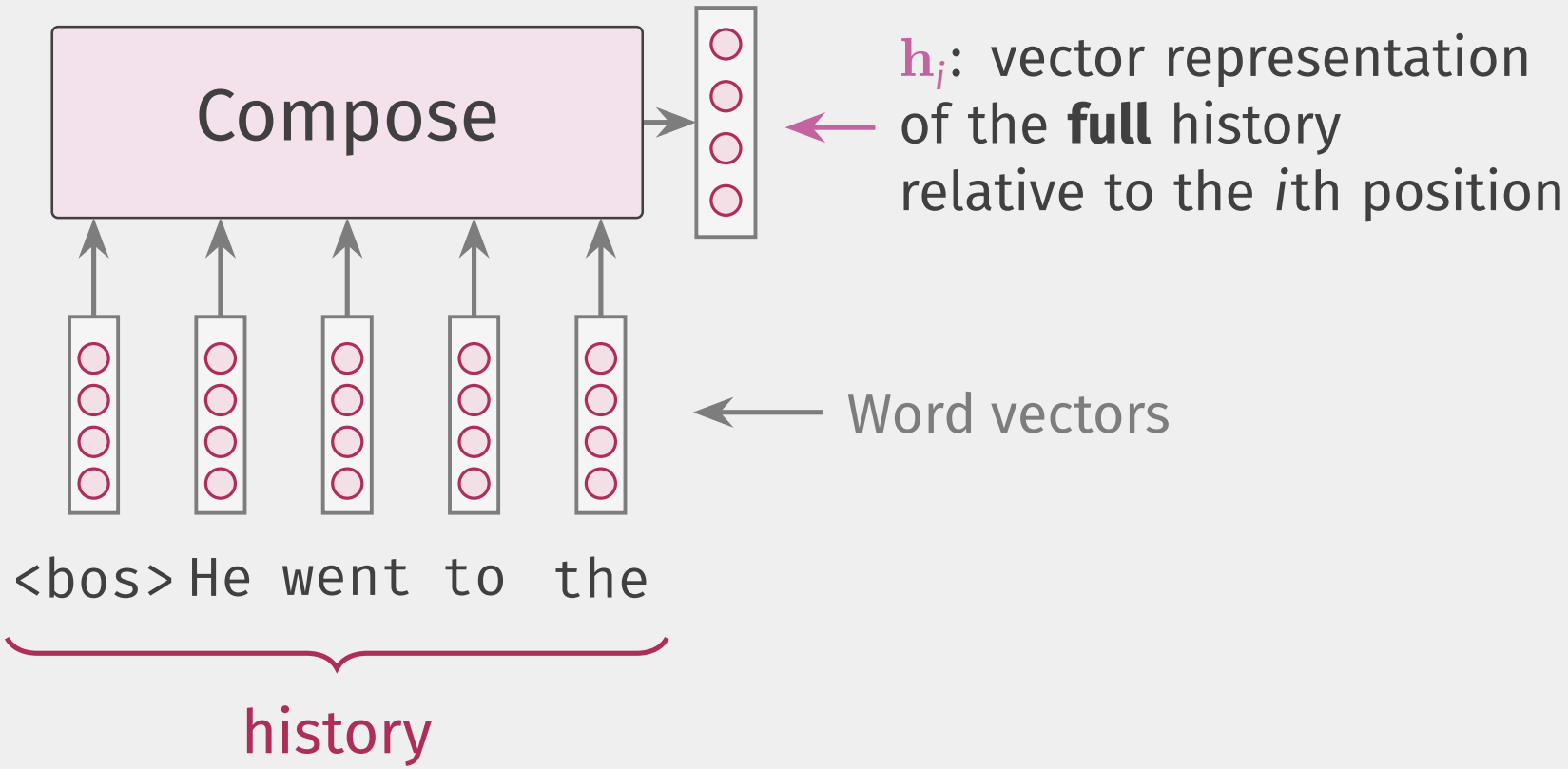
But we are still ignoring useful data:



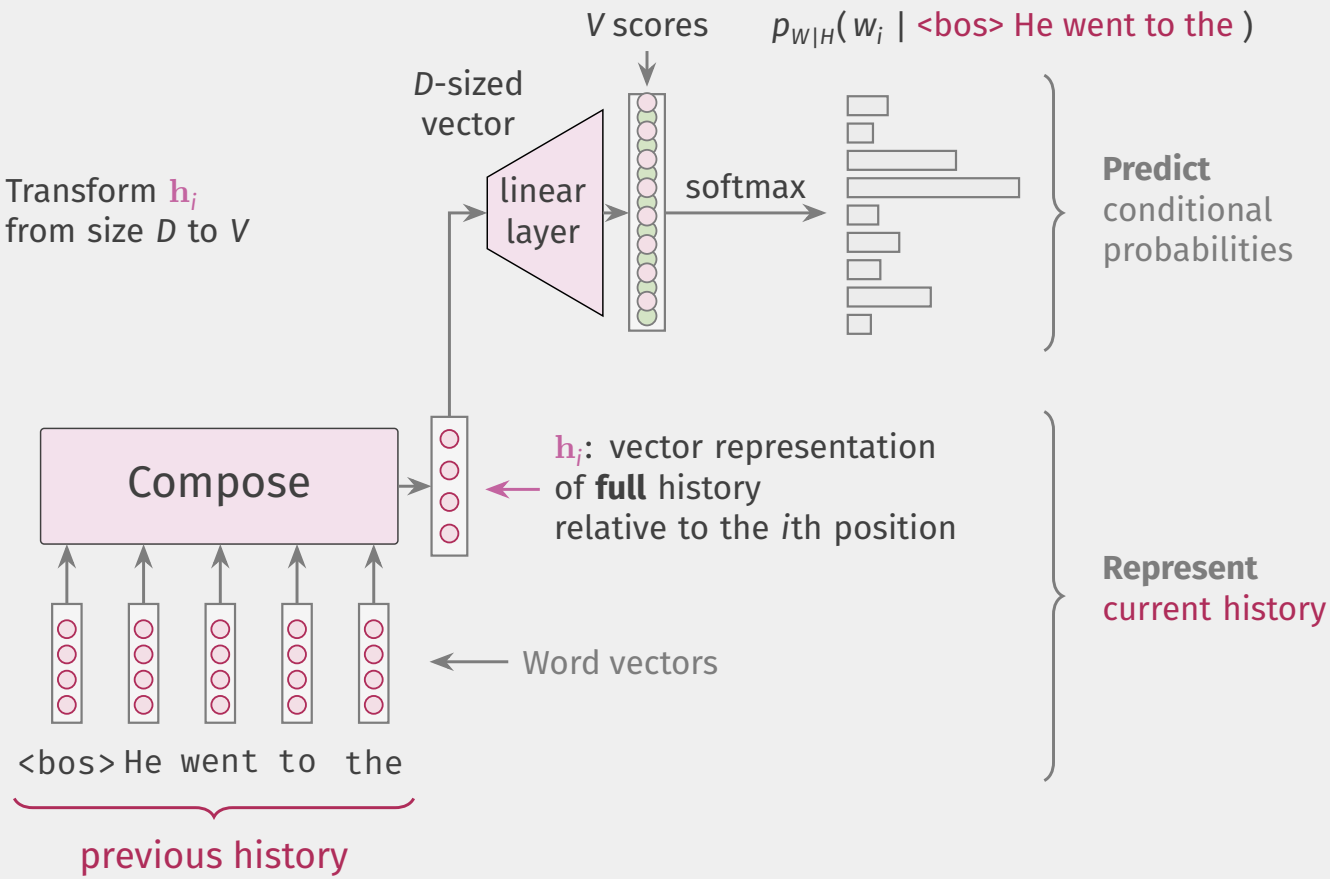
Discussion

This is considerably better than those tabular CPDs.

What stops us from using the complete history?

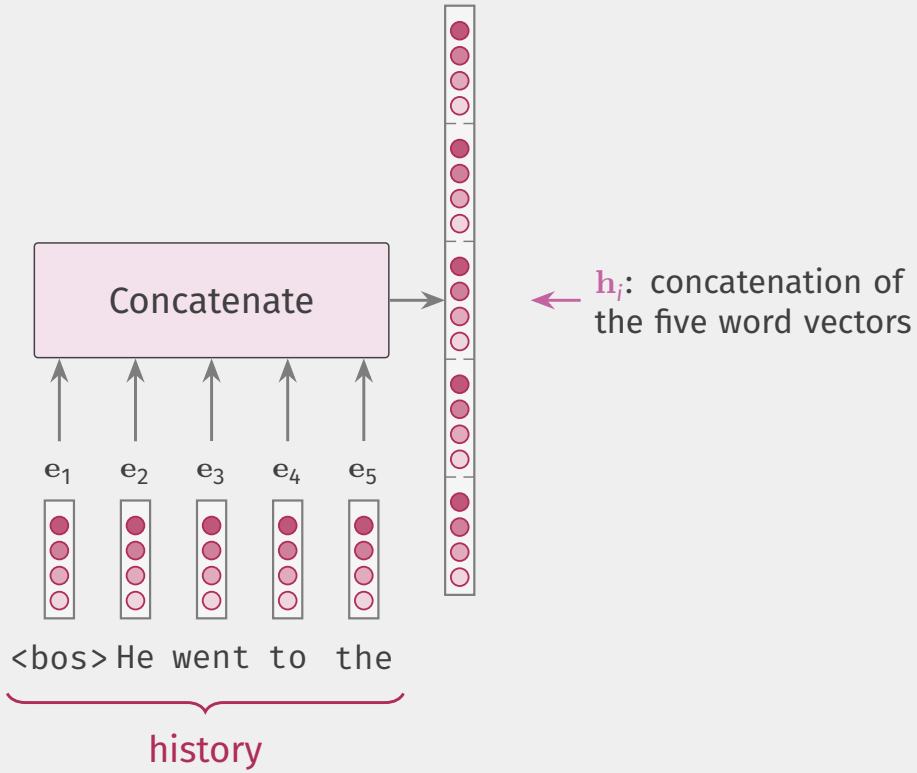


No Markov Assumptions



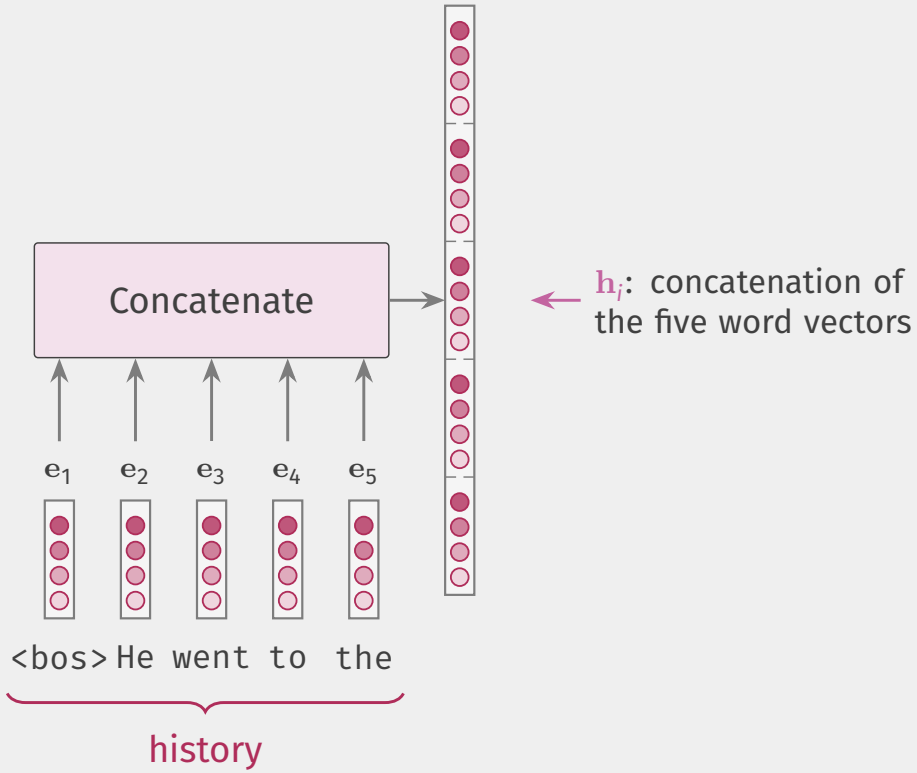
No matter the history's length, the final transformation to V scores requires $\mathbf{h}_i$ to be of a fixed, pre-specified dimensionality.

Concatenation



Concatenation preserves order,

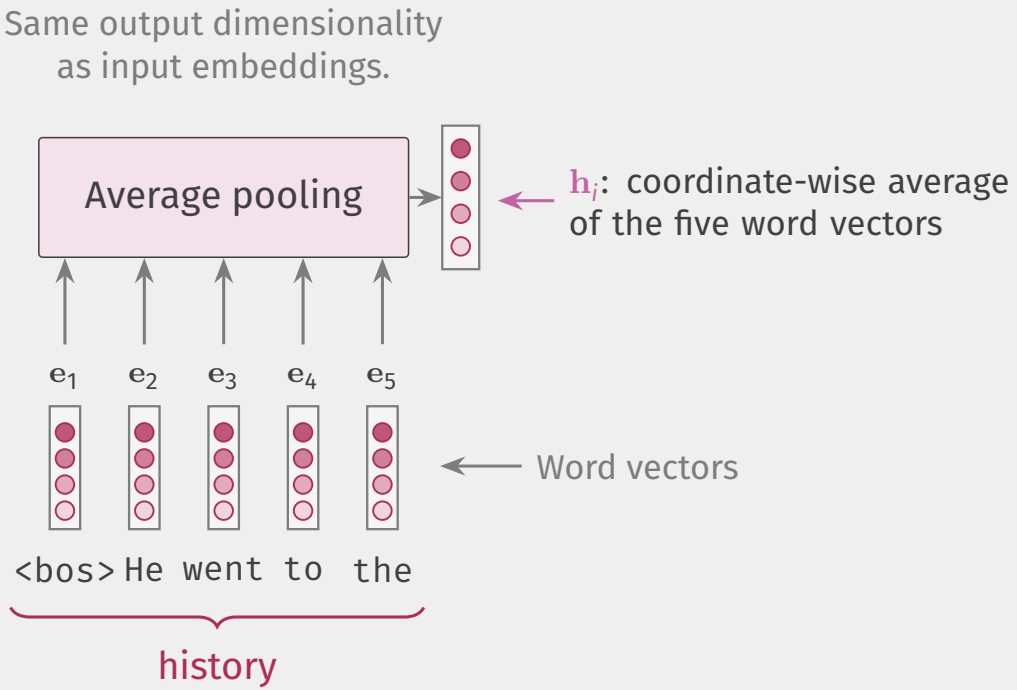
Concatenation



Concatenation preserves order, but dimensionality grows with history length.

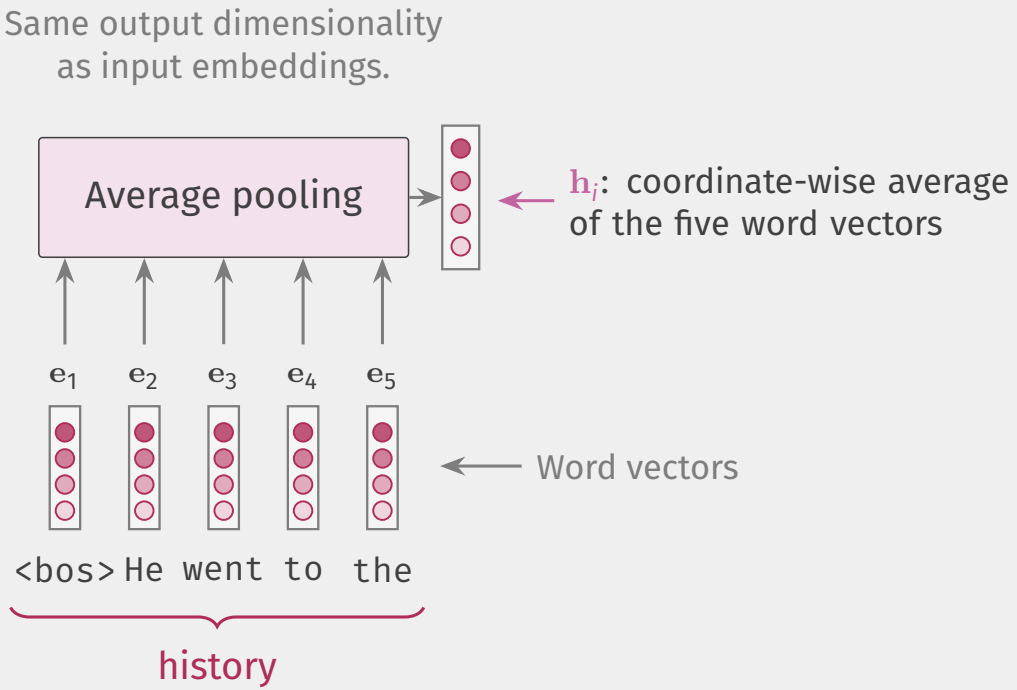
NOT AN OPTION!

Pooling



Pool from the variable-length history:

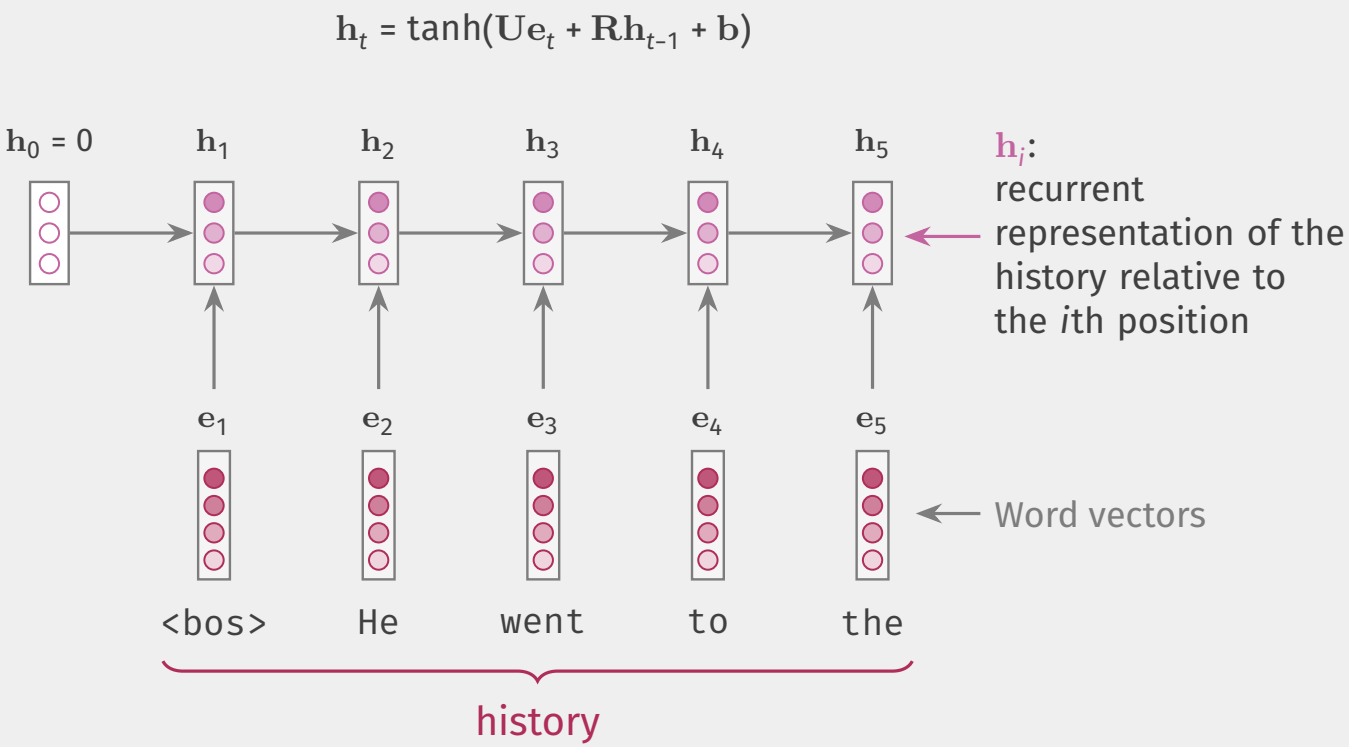
Pooling



Pool from the variable-length history: dimensionality stays constant, but order is lost. Not a great option!

Recurrent Composition

A recurrent neural network (RNN) has a notion of **state**. This state is updated iteratively, as inputs are presented in order.



An update consists of a feed-forward-like computation with parameters $\mathbf{U} \in \mathbb{R}^{H \times I}, \mathbf{R} \in \mathbb{R}^{H \times H}, \mathbf{b} \in \mathbb{R}^{H}$ that are reused at each step.

RNNs

RNNs are powerful tools, which dominated sequence modelling for nearly a good decade.

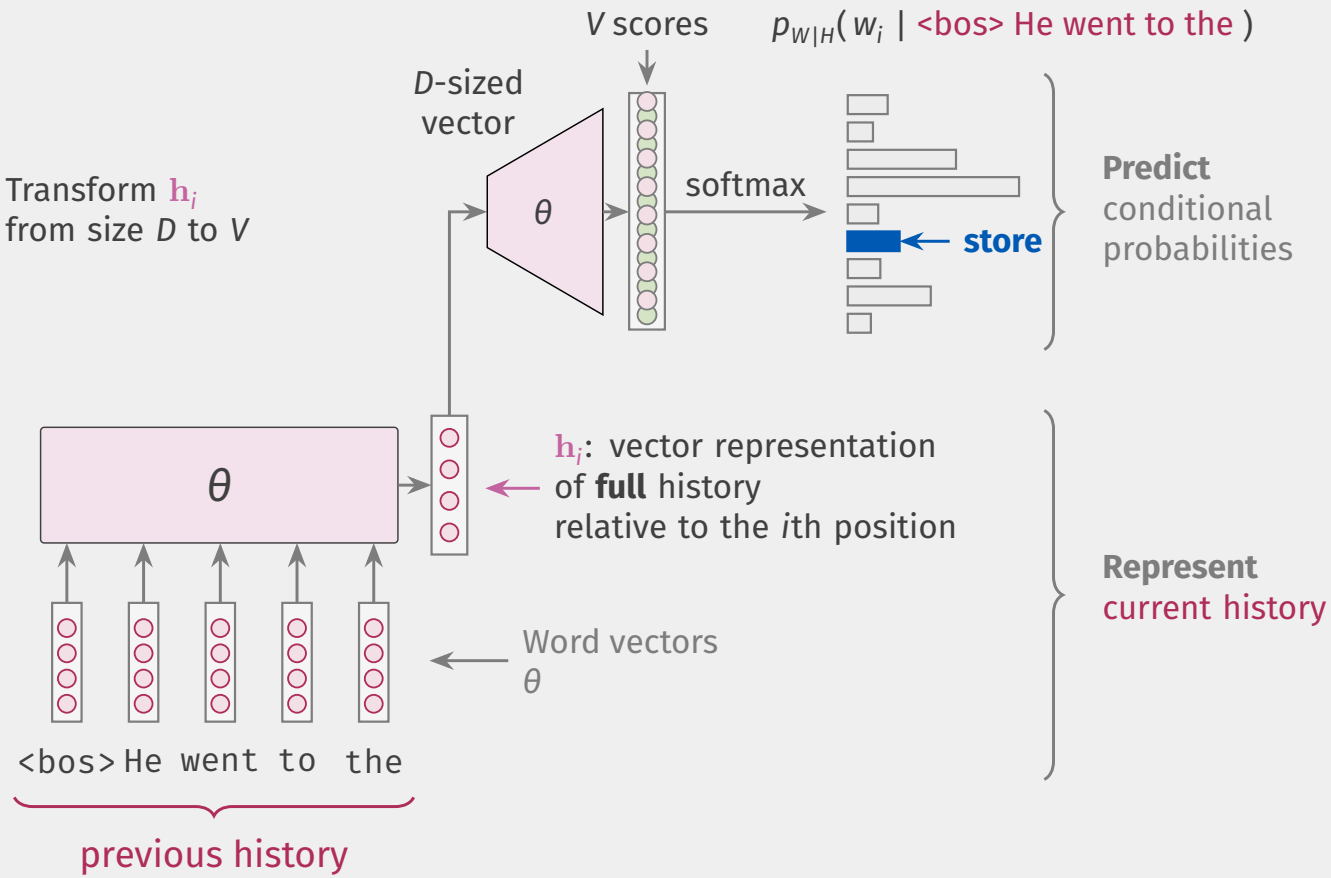
Basic RNNs have some problems

- over time, they struggle to ‘remember’ inputs
- during training, they lead to exploding and/or vanishing gradients

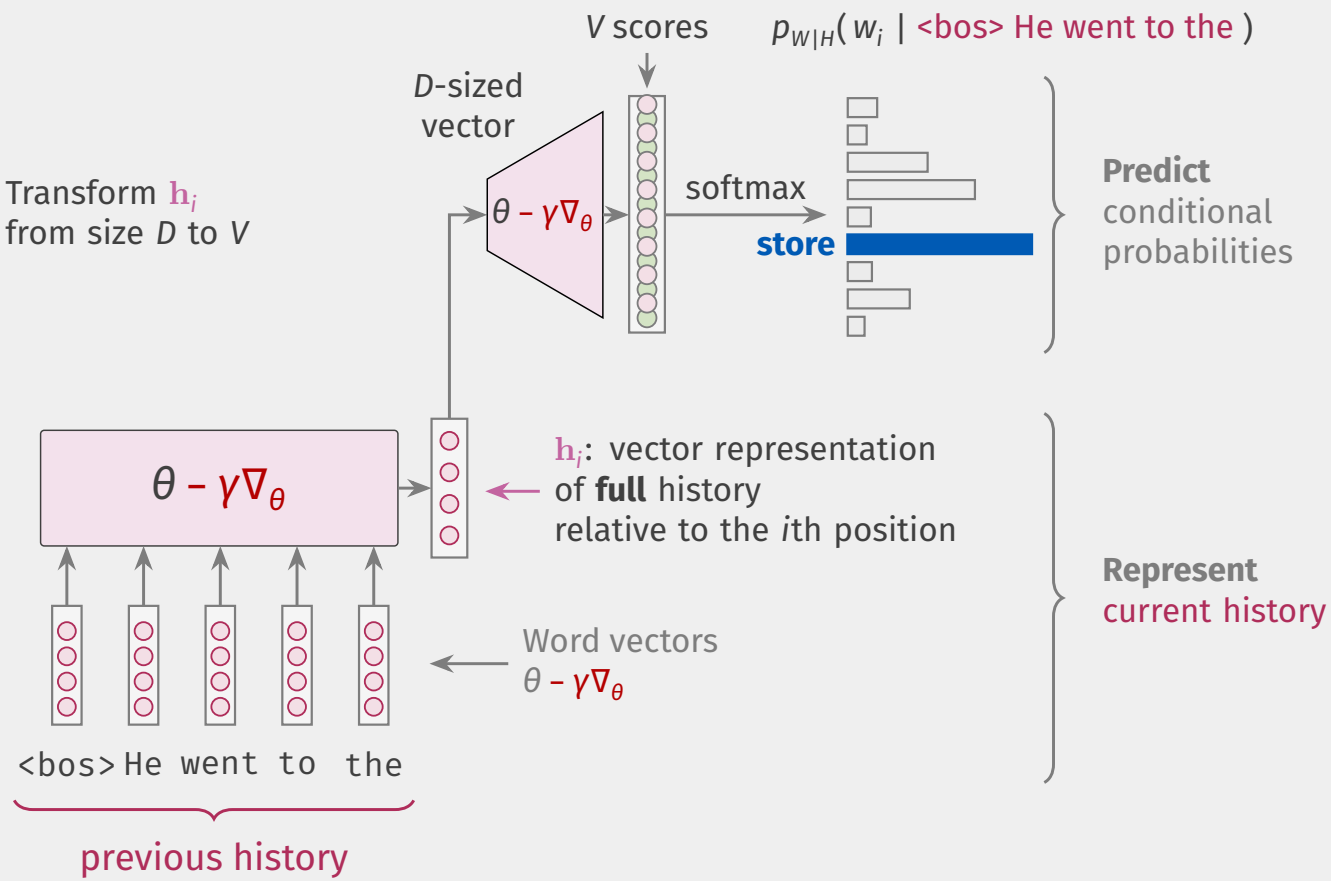
Improved RNN cells, such as GRUs and LSTMs (more about them in HC3b and in tutorial 2), fixed these issues.[10]

[10] TODO: cite some GRU/LSTM papers.

Parameter Estimation



As in the case of text classification, our goal is to minimise $-\log p(w_i|w_{<i};\theta)$ as a function of our choice of θ.



As before, we rely on auto-diff to get the objective's gradient and update the parameters in the direction of steepest descent.

In reality, our objective collects contributions from **all token positions**:

$$\ell(\theta | w_{1:l}) = -\log p_X(w_{1:l}; \theta)$$

In reality, our objective collects contributions from **all token positions**:

$$\ell(\theta|w_{1:l}) = -\log p_X(w_{1:l};\theta) = -\sum_{i=1}^{l} \log p_{W|H}(w_i|w_{<i};\theta)$$

In reality, our objective collects contributions from **all token positions**:

$$\ell(\theta|w_{1:l}) = -\log p_X(w_{1:l};\theta) = -\sum_{i=1}^{l} \log p_{W|H}(w_i|w_{<i};\theta)$$

Not only that, we use a batch B of texts. The loss is then the average for the entire batch:

$$\ell(\theta|B) = \frac{-1}{|B|} \sum_{x\in\mathcal{B}} \log p_X(x;\theta)$$

as its gradient is then an unbiased estimator of $\nabla_\theta \ell(\theta|\mathcal{D})$, where $\mathcal{D}$ is the entire training corpus.

Regularisation

The usual options

- size of hidden states
- weight decay (L2 penalty on weights)
- dropout (but don't use it in the recurrent cell; tutorial 2 has more information on this)

Summary

So long as we parameterise the network carefully as we did, there's nothing new to parameter estimation

- collect contributions to the loss
  - sum log probs per token position of each text
  - average the contributions of the texts in a batch
- let autodiff compute the gradient
- let optimiser perform update

Some important things about RNNs

- basic RNN cells are too simple (use GRU or LSTM instead)
- dropout makes no sense in the recurrent cell (see tutorial 2 for the right way to do it).

Evaluation

We assess the average surprisal (negative log probability) that our model assigns to heldout texts $\{x^{(1)}, \ldots, x^{(S)}\}$:

$$\frac{-1}{S} \sum_{s=1}^{S} \log p_X(x^{(s)}) \tag{5}$$

For ease of interpretation, we re-express it in terms of **perplexity per token**, a measure of average confusion.[11]

[Section 3.3 of textbook]

[11] If perplexity per token is 5, on average across histories, the model's uncertainty over the next token spreads over 5 candidates from the vocabulary.

Extrinsic

Plug the LM in a task (e.g., autocomplete) and measure the performance in that task.

We will be in a better position to cover extrinsic tasks in module 5, when we discuss sequence-to-sequence models.

Statistical

Compare how the statistics of generated text **distribute** in relation to statistics of observed text. See [2] and [1].

[Read section 3.4 of textbook for more on sampling]

Be Critical

Your statistical model is as good as your statistical assumptions, your estimation procedure, and the data you use to fit it.

- Most assumptions are wrong or insufficient.
- Our estimation procedures are based on crude statistics.
- Any dataset (however large) is at best a snippet of language production by some groups of speakers.

Models are not trained to **understand**, they are not trained to **respond**, they are not trained to **comply** with the values of the humans using it, they are not trained to produce **factually correct** text; they are trained such that their samples **look like** they could have been found in the training data.

What Next?

Self-study

- Sections 3.2, 3.3 and 3.4 of textbook; section 3.1 is useful and didactic, even though we do not focus much on NGram LMs; you can safely skip everything from and including section 3.6.
  `https://web.stanford.edu/~jurafsky/slp3/3.pdf`
- Sections 14.1 and 14.2 of textbook
  `https://web.stanford.edu/~jurafsky/slp3/14.pdf`

Deadlines

- TBA

Next module: Meaning (HC3a with Martha and HC3b with Vera)

References I



Mario Giulianelli, Joris Baan, Wilker Aziz, Raquel Fernández, and Barbara Plank.
What comes next? evaluating uncertainty in neural text generators against human production variability.
In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14349–14371, Singapore, December 2023. Association for Computational Linguistics.



Clara Meister and Ryan Cotterell.
Language model evaluation beyond perplexity.
In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 5328–5339, Online, August 2021. Association for Computational Linguistics.